



MILESTONE 1 — DATA MODEL SPECIFICATION

SupportPilot

MongoDB Collections, Embedding Strategy & Indexes for Weeks 1-2

DOCUMENT TYPE	Data model specification — collections, document shapes, indexes, decisions
PROJECT CODE	SP-ATR-001 / M1-DATA
DATABASE	MongoDB 7.x (Atlas or self-hosted replica set)
SCOPE	Milestone 1 collections only — intake, classification, severity, priority, SLA
PREREQUISITE	Read section 1 before writing any code. It affects your framework choice.

1. Decide This First: Django + MongoDB

The blueprint specified Django + PostgreSQL. Switching to MongoDB removes most of what Django was chosen for.

Django's ORM, migrations, admin site, and the built-in auth and permissions system are all built on SQL. None of them work against MongoDB. If the team writes `models.Model` classes expecting MongoDB underneath, they will hit this on day 2, not day 10.

1.1 The three realistic options

Option	What you use	Consequence
A. FastAPI + Beanie <i>Recommended</i>	FastAPI for the Core API, Beanie ODM (async, Pydantic-based) over Motor	Clean fit. Pydantic models double as API schemas and documents — one definition instead of three. You lose Django admin, so build the two or three admin screens yourself. Also unifies the stack: the AI service is already FastAPI.
B. Django + MongoEngine	Django for routing/views, MongoEngine as the ODM	Works, and keeps DRF serializers and views. But no Django admin, no migrations, no <code>django.contrib.auth</code> against Mongo — you write auth yourself. You keep Django's shape while losing its substance.
C. Hybrid	PostgreSQL for users/auth/config, MongoDB for tickets	Django admin and auth keep working; tickets go to Mongo. Two databases to operate, back up and keep consistent. Justifiable only if Django admin is genuinely required.

Recommendation: Option A. Your AI service is already FastAPI and Pydantic; using the same stack for the Core API means one mental model, one set of schemas, and no impedance mismatch. The document shapes below are written as Beanie/Pydantic documents but work unchanged with MongoEngine or raw PyMongo.

If MongoDB was chosen because of the vector search in Milestone 2

That is a sound reason. MongoDB Atlas Vector Search handles the RAG requirement natively, so you do not need a separate vector database. This document assumes that path and section 8 shows where the M2 collections attach.

2. Embedding Strategy

The single decision that determines whether this model performs well: what gets embedded inside a ticket document, and what lives in its own collection. The rule is **bounded and always read together → embed; unbounded or independently queried → reference.**

Data	Decision	Reasoning
Current classification	Embed	Exactly one per ticket, always displayed with it, read on every list and detail query. Embedding removes a join from the hottest query in the system.
Status history	Embed	Bounded — a ticket passes through perhaps 5–10 states. Always shown on the detail timeline.
Attachments metadata	Embed	Capped at 5 by the form. Files themselves live in object storage; only the metadata is embedded.
SLA state	Embed	One per ticket, needed for queue ordering. Embedding lets you index on <code>sla.resolution_due_at</code> directly.
Denormalised requester summary	Embed	Name, email, department, site copied onto the ticket. Prevents a <code>\$lookup</code> on every list render. See section 2.1.
Comments	Reference	Unbounded — a long-running ticket can accumulate hundreds. Embedding risks the 16 MB document limit and forces the whole array into memory on every read.
Classification history / overrides	Reference	This is your training dataset. It needs independent querying, aggregation and export, and it grows without limit.
Users, teams, taxonomy, SLA policies	Reference	Shared master data, edited independently, referenced by many tickets.
Activity log	Reference	Append-only, high volume, queried by actor and date rather than by ticket.

2.1 The denormalisation rule

Copy `requester.name`, `requester.email`, `requester.department` and `requester.site` onto every ticket at creation time. This is duplicated data and that is intentional — the ticket list is the most-hit query in the application and it must not fan out to the users collection for every row.

The cost is staleness: if someone changes department, old tickets keep the old value. That is **correct behaviour** for a ticket record — it captures who they were when they raised it. For anything that must stay current, read through the `requester.user_id` reference.

3. Milestone 1 Collections

users

Identity, role and profile. Referenced by tickets, comments, overrides.

```
{
  _id: ObjectId,
  email: "priya.sharma@company.com", // unique index
  full_name: "Priya Sharma",
  password_hash: "$argon2id$...", // null when SSO-only
  role: "EMPLOYEE", // EMPLOYEE|AGENT|ENGINEER|LEAD|KNOWLEDGE_MGR|ADMIN|SERVICE

  department_id: ObjectId,
  department_name: "Finance", // denormalised for display
  site_id: ObjectId,
  site_name: "Chennai – DLF IT Park",
  is_vip: false, // drives severity override rule
  is_active: true,
  mfa_enabled: false,
  last_login_at: ISODate,
  created_at: ISODate,
  updated_at: ISODate
}
```

Indexes: {email: 1} unique · {role: 1, is_active: 1} · {department_id: 1}

tickets

The central collection. Everything embedded here is read on the same screen.

```
{
  _id: ObjectId,
  ticket_number: "IT-2026-004521", // unique, from counters
  subject: "VPN connection failing on corporate network",
  description: "Unable to connect to VPN since this morning...",
  description_clean: "unable to connect vpn since morning...", // post-preprocessing
  channel: "PORTAL", // PORTAL|EMAIL|API|SERVICE_DESK

  requester: { // DENORMALISED – see 2.1
    user_id: ObjectId,
    name: "Priya Sharma",
    email: "priya.sharma@company.com",
    department_id: ObjectId,
    department: "Finance",
    site: "Chennai – DLF IT Park",
    is_vip: false
  },
  on_behalf_of_user_id: null,

  // ---- form: impact section ----
  impact: {
    affected_scope: "TEAM", // SELF|TEAM|DEPARTMENT|ORG_WIDE
    work_blocked: "YES", // YES|PARTIALLY|NO
    self_urgency: "NORMAL", // LOW|NORMAL|HIGH (weak feature only)
    workaroud: false
  },

  // ---- form: context section ----
  context: {
    category_hint: null,
    affected_system: "Cisco AnyConnect",
    started_when: "TODAY",
    asset_tag: "LT-04821",
    contact_pref: "EMAIL",
    best_time: null
  },

  // ---- EMBEDDED: current classification ----
  classification: {
    category: "VPN",
    sub_category: "Connection failure",
    severity: "HIGH", // after overrides applied
    severity_model: "MEDIUM", // what the model actually said
    priority: "P2", // deterministic matrix result
    confidence: 0.92,
    path: "FAST", // FAST|LLM|FAST_FALLBACK|NONE
    model_version: "clf-v1.2-lgbm",
    prompt_version: null,
    latency_ms: 47,
    overrides_applied: ["WORK_BLOCKED_TEAM"],
    is_manual_override: false,
    classified_at: ISODate
  },

  // ---- EMBEDDED: SLA state ----
  sla: {
    policy_id: ObjectId,
```

```
calendar:      "IN_CHENNAI",
first_response_due_at: ISODate,
resolution_due_at:  ISODate,
first_response_at:   null,
breached_first_response: false,
breached_resolution: false
},

// ---- assignment & state ----
status:      "AI_PROCESSING",
assigned_to_id:   null,
assigned_team_id: ObjectId,
assigned_team_name:"Network Team",

// ---- EMBEDDED: bounded arrays ----
status_history: [
  { from: null, to: "NEW", actor_id: ObjectId, actor_name: "Priya Sharma",
    reason: null, at: ISODate },
  { from: "NEW", to: "CLASSIFIED", actor_id: null, actor_name: "System",
    reason: "auto-classification", at: ISODate }
],
attachments: [
  { file_key: "tickets/2026/08/abc.png", file_name: "vpn-error.png",
    mime: "image/png", size_bytes: 184320, uploaded_by: ObjectId,
    scan_status: "CLEAN", uploaded_at: ISODate }
],

// ---- flags & counters ----
comment_count:  0,
reopen_count:   0,
is_deleted:     false,
duplicate_of_id: null,

created_at: ISODate,
updated_at: ISODate
}
```

Indexes for tickets

Index	Serves
{ticket_number: 1} unique	Lookup by human reference
{status: 1, "sla.resolution_due_at": 1}	Queue ordering by time-to-breach. The most important index in the system.
{"requester.user_id": 1, created_at: -1}	Portal "my tickets" view
{assigned_to_id: 1, status: 1}	Agent's personal queue
{assigned_team_id: 1, status: 1, "classification.priority": 1}	Team queue with priority sort
{"classification.category": 1, created_at: -1}	Category filter on the master list
{created_at: -1}	Default list ordering, cursor pagination
{subject: "text", description: "text"}	Keyword search. Weight subject higher: {weights: {subject: 3, description: 1}}
{status: 1} partial, where status ∈ NEW/CLASSIFIED/IN_PROGRESS	Hot working set. A partial index here is far smaller than a full one.

ticket_comments

Referenced, not embedded — unbounded growth.

```
{
  _id:      ObjectId,
  ticket_id: ObjectId,
  author_id: ObjectId,
  author_name: "Arun Kumar",           // denormalised
  body:      "Checked the firewall rules on ports 500 and 4500...",
  visibility: "INTERNAL",               // PUBLIC|INTERNAL
  source:    "HUMAN",                  // HUMAN|AI|EMAIL|SYSTEM
  created_at: ISODate
}
```

Indexes: {ticket_id: 1, created_at: 1} · {author_id: 1, created_at: -1}

classification_overrides

Your training dataset. Written every time an agent corrects the AI.

```
{
  _id:      ObjectId,
  ticket_id: ObjectId,
  ticket_number: "IT-2026-004521",

  predicted: { category: "NETWORK", sub_category: "Connectivity",
               severity: "MEDIUM", confidence: 0.64, path: "FAST",
               model_version: "clf-v1.2-lgbm" },
  corrected: { category: "VPN", sub_category: "Connection failure",
```

```

        severity: "HIGH" },

// snapshot of the text as classified – the label is worthless without it
subject_snapshot: "VPN connection failing on corporate network",
description_snapshot: "Unable to connect to VPN since this morning...",
features_snapshot: { department: "Finance", channel: "PORTAL",
                    affected_scope: "TEAM", work_blocked: "YES" },

corrected_by_id: ObjectId,
corrected_by_name: "Arun Kumar",
reason: "Cisco AnyConnect issue, not general network",
used_in_training: false,           // set true once consumed by a retrain
created_at: ISODate
}

```

Why the text snapshots matter

If you store only ticket_id and the corrected label, and someone later edits the ticket description, your training example silently becomes wrong. Snapshotting the exact text that was classified makes each override a self-contained, immutable training row. This collection is the reason the ReclassifyDialog is a Milestone 1 deliverable.

Indexes: {ticket_id: 1} · {used_in_training: 1, created_at: 1} · {"predicted.category": 1, "corrected.category": 1} for confusion analysis

category_taxonomy

Frozen for the milestone. Drives dropdowns and the LLM prompt.

```

{
  _id: ObjectId,
  code: "VPN",
  name: "VPN",
  description: "Virtual private network access and connectivity",
  sub_categories: [
    { code: "CONN_FAIL", name: "Connection failure" },
    { code: "CERT", name: "Certificate" },
    { code: "CLIENT_INST", name: "Client install" },
    { code: "TIMEOUT", name: "Timeout" }
  ],
  default_team_id: ObjectId,
  keywords: ["vpn", "anyconnect", "tunnel", "remote access"],
  training_count: 412,           // drives the "<30 examples → LLM" routing rule
  is_active: true,
  sort_order: 2
}

```

Indexes: {code: 1} unique · {is_active: 1, sort_order: 1}

sla_policies

```

{
  _id: ObjectId,
  priority: "P2",           // unique
  first_response_minutes: 60,
  resolution_minutes: 480,
  calendar_type: "BUSINESS", // BUSINESS|TWENTYFOUR_SEVEN
  is_active: true,
  effective_from: ISODate
}

```

business_calendars

Per site. Without this, a Friday 5pm P3 is "due" on Friday night.

```

{
  _id: ObjectId,
  code: "IN_CHENNAI",
  name: "India – Chennai",
  timezone: "Asia/Kolkata",
  working_days: [1,2,3,4,5], // Mon–Fri (0 = Sunday)
  work_start: "09:30",
  work_end: "18:30",
  holidays: [
    { date: ISODate("2026-08-15"), name: "Independence Day" },
    { date: ISODate("2026-10-20"), name: "Deepavali" }
  ]
}

```

departments · teams · sites

Small master-data collections. Keep them separate — they are edited independently.

```

// departments
{ _id, code: "FIN", name: "Finance", manager_id: ObjectId, is_active: true }

```

```
// teams
{ _id, code: "NET", name: "Network Team", department_id: ObjectId,
  queue_key: "network", member_ids: [ObjectId, ...],
  is_escalation_target: true, is_active: true }

// sites
{ _id, code: "MAA01", name: "Chennai — DLF IT Park", country: "IN",
  calendar_code: "IN_CHENNAI", is_active: true }
```

applications

Powers the `affected_system` autocomplete and a classifier feature.

```
{ _id, name: "Cisco AnyConnect", aliases: ["anyconnect", "vpn client"],
  category_hint: "VPN", owner_team_id: ObjectId, is_active: true }
```

Indexes: `{name: "text", aliases: "text"}` for autocomplete

activity_logs

Append-only. Never updated, never deleted except by retention.

```
{
  _id:      ObjectId,
  actor_id: ObjectId,
  actor_name: "Arun Kumar",
  action:    "CLASSIFICATION_OVERRIDDEN",
  entity_type: "TICKET",
  entity_id: ObjectId,
  before:     { category: "NETWORK", severity: "MEDIUM" },
  after:      { category: "VPN", severity: "HIGH" },
  ip:         "10.4.22.118",
  request_id: "req_8f3a2c...",
  created_at: ISODate
}
```

Indexes: `{entity_type: 1, entity_id: 1, created_at: -1}` · `{actor_id: 1, created_at: -1}` · TTL index on `created_at` at the retention period, or a time-series collection.

counters

Small but essential — this is how you generate ticket numbers safely.

```
{ _id: "ticket_number_2026", seq: 4521 }
```

Do not generate ticket numbers with `count() + 1`

Under any concurrency that produces duplicates. Use an atomic `findOneAndUpdate` with `$inc: {seq: 1}` and `returnDocument: "after"`. MongoDB guarantees the increment is atomic; two simultaneous submissions get 4522 and 4523, never both 4522. Combine with a unique index on `ticket_number` as a second line of defence.

4. Schema Validation

MongoDB is schemaless by default. That is not an advantage here — a typo in a field name silently creates a new field instead of raising an error. Apply `$jsonSchema` validators on every collection at creation time.

```
db.createCollection("tickets", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["ticket_number", "subject", "description", "channel",
              "requester", "impact", "status", "created_at"],
    properties: {
      ticket_number: { bsonType: "string", pattern: "^IT-[0-9]{4}-[0-9]{6}$" },
      subject:       { bsonType: "string", minLength: 5, maxLength: 150 },
      description:   { bsonType: "string", minLength: 20 },
      channel:       { enum: ["PORTAL", "EMAIL", "API", "SERVICE_DESK"] },
      status:        { enum: ["NEW", "CLASSIFIED", "AI_PROCESSING", "PENDING_REVIEW",
                              "ASSIGNED", "IN_PROGRESS", "WAITING_ON_USER",
                              "ESCALATED", "AI_RESOLVED", "RESOLVED", "REOPENED",
                              "CLOSED", "FAILED", "CANCELLED"] },
      classification: { bsonType: "object", properties: {
        severity: { enum: ["LOW", "MEDIUM", "HIGH", "CRITICAL"] },
        priority: { enum: ["P1", "P2", "P3", "P4"] },
        confidence: { bsonType: "double", minimum: 0, maximum: 1 },
        path:       { enum: ["FAST", "LLM", "FAST_FALLBACK", "NONE"] }
      } }
    }
  } },
  validationLevel: "strict",
  validationAction: "error"
})
```

Keep the enum lists in **one** Python module and generate the validators from it, so the database, the Pydantic models and the frontend constants cannot drift apart.

5. Transactions & Consistency

Operation	Approach
Create ticket	Two writes: increment counter, insert ticket. Wrap in a transaction (requires a replica set — a standalone mongod cannot do transactions, so configure a single-node replica set even in development).
Apply classification	Single updateOne setting classification, sla, status and pushing to status_history atomically. One document, one atomic write — no transaction needed. This is where the embedded design pays off.
Override classification	Two collections: update the ticket, insert the override record. Use a transaction — a training label without its ticket update is a silent inconsistency.
Add comment	Insert comment, \$inc the ticket's comment_count. Transaction, or accept eventual drift and recompute nightly.
Status transition	Guard with a conditional update: updateOne({_id, status: expectedFrom}, {\$set: {status: to}, \$push: {...}}). If modifiedCount is 0, the transition raced with another and you return 409. This gives you optimistic concurrency without a transaction.

6. Query Patterns to Build Against

Screen	Query shape
Agent queue (SLA order)	<code>find({assigned_team_id, status: {\$in: [...]}}).sort({"sla.resolution_due_at": 1})</code> — served entirely by the compound index, no in-memory sort.
Master ticket list	<code>find(...filters).sort({created_at: -1}).limit(25)</code> with cursor pagination on <code>created_at</code> plus <code>_id</code> as tiebreaker. Avoid <code>skip()</code> — it degrades linearly.
Portal my tickets	<code>find({"requester.user_id": userId, is_deleted: false}).sort({created_at: -1})</code>
Ticket detail	One <code>findOne</code> on tickets (classification, SLA, history, attachments all embedded) plus one <code>find</code> on comments. Two queries for the whole screen.
Duplicate check	<code>find({"requester.user_id", status: {\$nin: closed}, created_at: {\$gt: 7 days ago}})</code> then score in application code. Small candidate set, no index gymnastics needed.
Dashboard counters	aggregate with <code>\$match</code> on today plus <code>\$group</code> by category. At M1 volume this runs in milliseconds; move to pre-computed rollups in M4.
Training set export	<code>find({used_in_training: false})</code> on <code>classification_overrides</code> , plus a bulk read of historical tickets.

7. What Milestone 1 Does NOT Need

Do not create these yet. Listing them so nobody builds ahead of scope:

Collection	Milestone
knowledge_articles, article_chunks	M2 — RAG
ticket_responses, response_citations, resolution_feedback	M2 — resolution generation
agent_runs, agent_steps	M3 — multi-agent
escalations, routing_rules	M3 — escalation engine
jira_links, email_logs, email_templates	M3 — integrations
daily_rollups, ai_quality_rollups	M4 — analytics

8. Forward Compatibility

8.1 Vector search lands in Milestone 2

If you are on Atlas, `article_chunks` gets an Atlas Vector Search index on an `embedding` field. Nothing in the M1 model needs to change to accommodate it — that is one of the genuine advantages of the MongoDB choice here.

```
// M2 – do not create yet
{ _id, article_id: ObjectId, chunk_index: 0,
  content: "To resolve VPN timeout errors, first verify...",
  heading_path: "VPN Troubleshooting > Timeout Errors",
  embedding: [0.021, -0.118, ...], // 1024 floats
  token_count: 640, created_at: ISODate }
```

8.2 Two things to get right now so M2-M4 do not require a migration

Decision	Why now
Keep classification as an embedded object , not a top-level flat set of fields	In M3 the multi-agent flow adds fields to it. An object absorbs new keys; twelve flat <code>classification_*</code> fields become a mess.
Store <code>model_version</code> and <code>prompt_version</code> on every classification from day one	Without it you cannot answer "which model produced this?" three months from now, and your M4 quality analysis has no way to compare model generations.

9. Setup Checklist

☐	Item
☐	Framework decision made and recorded (section 1) before any model code is written
☐	MongoDB running as a replica set, even in local development — transactions require it
☐	All 11 M1 collections created with <code>\$jsonSchema</code> validators, <code>validationAction: "error"</code>
☐	Enums defined once in Python; validators and frontend constants generated from that source
☐	Every index in section 3 created and verified with <code>explain()</code> showing <code>IXSCAN</code> , not <code>COLLSCAN</code>
☐	Ticket numbering uses atomic <code>\$inc</code> ; concurrency-tested with 100 simultaneous creates
☐	Status transitions use conditional updates and return 409 when <code>modifiedCount</code> is 0
☐	Seed script loads users, departments, teams, sites, taxonomy, SLA policies, calendars
☐	Historical ticket import script written — needed for the M1 training and hold-out sets
☐	Backup configured and one restore rehearsed before the milestone review